Heart Disease Risk Prediction: MLOps Assignment 01 (AIMLCZG523)

1. Data acquisition and EDA (5 marks)

• Source: UCI Machine Learning Repository, Heart Disease dataset (Cleveland subset), downloaded by data/download_data.py.

• Shape: 303 rows, 13 features plus the target. Missing values: `ca` (4), `thal` (2), encoded as `?` in the source.

• Target: `num > 0` is mapped to 1 (heart disease). Classes: 164 no disease, 139 disease (45.9% positive).

• No duplicate rows.

• Figures: reports/figures/histograms_by_target.png, reports/figures/class_balance.png, reports/figures/correlation_heatmap.png.

• Summary statistics: reports/eda_summary.txt.

Key Observations:

The exploratory data analysis reveals several important patterns in the heart disease dataset:

1. **Feature Distributions**: Histograms show that continuous features (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`) follow approximately normal distributions with some right skew. Age ranges from 29 to 77 years, providing good representation of the patient population.

2. **Disease Association**: Features strongly associated with heart disease include:

- Higher `oldpeak` (ST depression) values correlate with increased disease presence

- Lower `thalach` (maximum heart rate achieved) is indicative of disease

- Chest pain type (`cp`) shows categorical patterns with specific types more prevalent in disease cases

- Thalassemia type (`thal`) demonstrates strong categorical relationship with disease

3. **Class Balance**: The dataset shows a near-balanced class distribution (45.9% positive, 54.1% negative), reducing the need for aggressive class reweighting while maintaining good discriminative power.

4. **Missing Data**: Missing values in `ca` (4 rows) and `thal` (2 rows) represent only 1.98% of the dataset, manageable through median imputation for numeric and mode imputation for categorical features.

5. **Correlation Patterns**: Correlation analysis shows moderate inter-feature dependencies without severe multicollinearity, indicating good feature diversity for model learning.

2. Feature engineering and model development (8 marks)

• Preprocessing (src/preprocessing.py): numeric features (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`) use median imputation and standard scaling. Categorical features (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`) use most-frequent imputation and one-hot encoding, with unseen categories ignored at inference.

• Candidate models (src/models.py): logistic regression and random forest, each in a scikit-learn Pipeline with a tuning grid.

• Tuning: 5-fold stratified cross-validation with `GridSearchCV` on the training split (80%). Selection metric: ROC-AUC.

| Model | CV ROC-AUC | Test ROC-AUC | Test recall | Test precision |

| Logistic regression (selected) | 0.903 ± 0.017 | 0.965 | 0.929 | 0.839 |

| Random forest | 0.902 ± 0.033 | 0.948 | 0.893 | 0.806 |

Model Selection Justification:

Logistic Regression was selected as the final model for the following reasons:

1. **Comparable Performance**: Both models achieve excellent ROC-AUC scores (LR: 0.965, RF: 0.948), but logistic regression demonstrates superior test-set ROC-AUC despite slightly lower recall.

2. **Lower Variance**: Logistic regression shows more consistent performance across CV folds (std: ±0.017 vs ±0.033 for random forest), indicating better generalization and robustness.

3. **Interpretability**: In clinical applications, interpretable models are preferred. Logistic regression provides clear feature coefficients and probability estimates, enabling clinicians to understand prediction rationale.

4. **Inference Speed**: Logistic regression offers O(n) prediction complexity vs O(n*log(n)) for random forest, critical for real-time clinical deployment.

5. **Hyperparameter Stability**: Selected parameters (C=0.1, class_weight=balanced) demonstrate robust performance, with balanced precision (83.9%) and recall (92.9%) - appropriate for medical diagnosis where missing disease cases (false negatives) is costly.

6. **Production Suitability**: Smaller model size, lower memory footprint, and easier containerization make logistic regression ideal for cloud deployment and resource-constrained environments.

3. Experiment tracking with MLflow (5 marks)

• Tracking backend: SQLite at `mlflow.db` (see src/config.py).

• Experiment: `heart-disease-risk`. Each model has a parent run with nested runs for each tuning candidate.

• Logged: parameters, CV metrics (mean and std), test metrics, confusion matrix and ROC figures, cross-validation results CSV, and the fitted model.

• Screenshot: MLflow runs dashboard showing experiment tracking with complete parameter and metric logging for both candidate models.

4. Model packaging and reproducibility (7 marks)

• Model saved as `models/model.joblib` and logged as an MLflow model (cloudpickle serialization).

• Dependencies pinned in requirements.txt and requirements-dev.txt.

• Fixed random seed (`RANDOM_STATE=42`) and stratified splits.

• Configuration centralised in src/config.py.

• Reproduce with the commands in README.md.

5. CI/CD and automated testing (8 marks)

• Tests in tests/: data loading and cleaning, preprocessing behaviour, model validity, save and load round trip, inference, and API endpoints (validation, optional fields, metrics, and logging). **20 tests pass locally and in CI/CD pipeline.**

• Linting with ruff to enforce code quality standards.

• GitHub Actions workflow .github/workflows/ci.yml with four jobs:

1. **Lint and unit tests**: Runs ruff linting and pytest (20/20 passing)

2. **EDA and model training**: Full pipeline execution with MLflow logging

3. **Docker build and smoke test**: Container build validation with health check and API endpoint tests

4. **Publish to GHCR**: Image publishing to GitHub Container Registry (requires credentials)

• Screenshot: All three critical CI/CD jobs passing (lint, train, docker), demonstrating reproducibility and production readiness.

6. Model containerization with Docker (5 marks)

• Dockerfile: Python 3.11 slim base image, pinned requirements for reproducibility, non-root user for security, health check endpoint, and uvicorn ASGI server entry point.

• docker-compose.yml orchestrates three services: API (uvicorn on port 8000), Prometheus monitoring (port 9090), and Grafana visualization (port 3000 with default admin/admin credentials).

• Multi-stage build optimization reduces image size while maintaining all production dependencies.

• Container successfully builds and runs in CI/CD pipeline with smoke tests validating `/health` and `/predict` endpoints.

7. Production deployment (7 marks)

• **Local Deployment**: Successfully deployed and tested on BITS Pilani Virtual Lab (Ubuntu Linux, Python 3.11)

- API running on http://localhost:8000

- Swagger UI documentation available at http://localhost:8000/docs

- Health check endpoint verified: http://localhost:8000/health

• **Kubernetes manifest** k8s/deployment.yaml: Production-ready Kubernetes deployment with:

- 2 replicas for high availability

- Readiness and liveness probes on `/health` endpoint

- Resource requests and limits for fair cluster scheduling

- LoadBalancer Service on port 80 for external traffic

- Image pull policy and restart policies configured for resilience

• **Cloud Deployment Ready**: Manifest compatible with any Kubernetes cluster (Minikube, AKS, GKE, EKS)

- Image available in GitHub Container Registry (ghcr.io)

- Supports horizontal pod autoscaling

- Implements graceful shutdown and health monitoring

• **Access Instructions**:

- Local: `http://localhost:8000`

- Kubernetes: `kubectl get svc heart-api` to obtain LoadBalancer EXTERNAL-IP

- Docker Compose: `docker compose up --build` for full stack

API Deployment Screenshots:

8. Monitoring and logging (3 marks)

• **Structured Logging**: Request logs captured to both stdout and persistent `logs/api.log` file containing:

- HTTP method and endpoint path

- Response status code and latency in milliseconds

- Client IP address for security auditing

- Standardized JSON format for easy parsing and analysis

• **Prometheus Metrics** exported at `/metrics` endpoint:

- `api_requests_total{method,endpoint,status}`: Total HTTP request counter by method, path, and status

- `api_request_latency_seconds`: Request latency histogram with quantile bucketing for SLO monitoring

- `api_predictions_total{label}`: Prediction counter by output label (useful for model performance drift detection)

• **Monitoring Stack**:

- Prometheus configuration monitoring/prometheus.yml scrapes API metrics every 15 seconds

- Grafana dashboard available through docker-compose for visualization and alerting

- Full stack deployable with: `docker compose up --build`

• **Production Observability**: Integrated logging enables:

- Real-time performance monitoring via Prometheus/Grafana

- Historical request analysis for anomaly detection

- Model performance tracking through prediction counters

- SLA compliance verification through latency histograms

9. Documentation and reflection (2 marks)

Comprehensive Documentation

Repository Documentation:

• README.md: Complete setup, reproduction, serving, Docker, Kubernetes, CI/CD, and monitoring instructions

• SETUP_LAB.md: Lab environment-specific deployment guide with step-by-step instructions

• DEPLOYMENT_EMAIL_PLAINTEXT.txt: Plain-text deployment guide for easy distribution

• Inline code documentation and docstrings throughout source files

• Structured project layout with clear separation of concerns

Key Achievements

■ **End-to-end MLOps Pipeline**: Complete data-to-deployment workflow with all production components

■ **Model Performance**: 96.5% test ROC-AUC on independent test set with clinical-grade metrics

■ **Automated Testing**: 20/20 unit tests passing, validating all critical components

■ **CI/CD Integration**: GitHub Actions pipeline ensuring code quality and reproducibility

■ **Production Deployment**: Docker containerization and Kubernetes manifests for scalable deployment

■ **Experiment Tracking**: Full MLflow integration for reproducibility and model governance

■ **Monitoring & Observability**: Prometheus metrics and structured logging for operational insights

Dataset Limitations

1. **Sample Size**: 303 rows is relatively small for deep learning; sufficient for traditional ML but limits model capacity

2. **Single-Site Data**: Cleveland subset only; geographic bias toward North American population and healthcare practices

3. **Class Imbalance**: While modest (45.9% positive), slightly favors negative class requiring careful threshold tuning

4. **Feature Representation**: Limited to common clinical measurements; newer biomarkers (troponin, NT-proBNP) not available

5. **Temporal Aspect**: Cross-sectional data without longitudinal follow-up limits disease progression prediction

Model Limitations

1. **Static Predictions**: Model provides point-in-time diagnosis without considering temporal disease progression

2. **Interpretability-Performance Trade-off**: Logistic regression prioritizes interpretability over potential nonlinear modeling capacity

3. **No Drift Detection**: Current system lacks automated detection of data/model drift in production

4. **Limited Explanability**: No SHAP/LIME integration for per-prediction feature attribution

Recommendations for Future Work

1. **Model Drift Monitoring**: Implement automated retraining triggers when prediction distribution shifts exceed thresholds

2. **A/B Testing Framework**: Deploy A/B tests comparing logistic regression against ensemble methods in production

3. **Feature Engineering**: Incorporate derived features (ratios, interactions) and domain expert knowledge

4. **External Validation**: Validate model on independent datasets (e.g., Vienna, Hungarian, Long Beach subsets)

5. **Explainability**: Add SHAP values and LIME explanations to API for clinician trust and regulatory compliance

6. **Real-time Retraining**: Implement online learning pipeline to continuously improve with new patient data

7. **Clinical Integration**: Develop web interface for direct EHR integration and clinical workflow optimization

8. **Fairness Analysis**: Audit model for demographic parity and ensure equitable performance across patient subgroups

References

• Detrano, R. et al. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. UCI Machine Learning Repository, Heart Disease dataset.

• Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

• MLflow documentation: https://mlflow.org/docs/latest/

• FastAPI documentation: https://fastapi.tiangolo.com/

• Kubernetes documentation: https://kubernetes.io/docs/

• Prometheus documentation: https://prometheus.io/docs/